

Object Init

Yes — this is a **very important C++ distinction**, especially before you learn polymorphism.

First, one correction: if `Animal` is an abstract class, you **cannot** create either of these directly. So let's assume `Animal` is a normal class.

1. `Animal a;`

```
Animal a;
```

This creates an **object directly**.

Conceptually:



`a` is the object itself.

You access it with `.`:

```
a.sound();
a.age;
```

Its lifetime is automatic: when the scope ends, `a` is destroyed.

```
void foo() {
    Animal a;

} // a automatically destroyed here
```

Use this when

You simply need an object and **don't need dynamic lifetime**.

This should generally be your default.

2. `Animal* a = new Animal();`

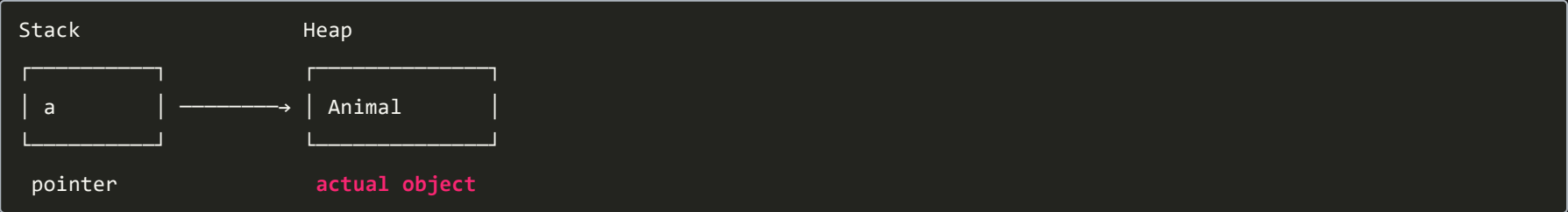
This is different.

```
Animal* a = new Animal();
```

Two things are happening:



Conceptually:



`a` is **NOT** the Animal object.

`a` is a **pointer to the Animal object**.

Therefore:

```
a->sound();
```

instead of:

```
a.sound();
```

3. The biggest difference: lifetime

Stack/local object

```
Animal a;
```

C++ automatically destroys it when it goes out of scope.

```
void foo() {  
    Animal a;  
  
} // automatically destroyed
```

Dynamically allocated object

```
Animal* a = new Animal();
```

You are responsible for destroying it:

```
delete a;
```

So:

```
Animal* a = new Animal();  
  
// use it  
  
delete a;
```

If you forget `delete`, you can create a **memory leak**.

4. But here's the REALLY important reason `new` appears in OOP

You'll often see:

```
Animal* a = new Dog();
```

This is where polymorphism comes in.

Suppose:

```
class Animal {  
public:  
    virtual void sound() {  
        cout << "Animal sound";  
    }  
};  
  
class Dog : public Animal {  
public:  
    void sound() override {  
        cout << "Bark";  
    }  
};
```

Now:

```
Animal* a = new Dog();
```

Look carefully:

Pointer type		Actual object
Animal*	→	Dog

Because `sound()` is virtual:

```
a->sound();
```

calls:

```
Dog::sound()
```

This is **runtime polymorphism**.

5. Why can't we just do this?

You might wonder:

```
Animal a = Dog();
```

You can technically write this, but it causes **object slicing**.

The `Dog` part specific to `Dog` gets sliced away, leaving an `Animal` object.

That's why polymorphism commonly uses:

```
Animal* a = new Dog();
```

or, much better in modern C++:

```
Animal* a = &dog;
```

or smart pointers:

```
unique_ptr<Animal> a = make_unique<Dog>();
```

6. One important modern C++ rule

Don't learn:

```
Animal* a = new Animal();  
...  
delete a;
```

as the preferred modern C++ style.

Usually prefer:

```
Animal a;
```

for a normal local object.

And when you genuinely need dynamic allocation:

```
unique_ptr<Animal> a = make_unique<Dog>();
```

because `unique_ptr` automatically handles destruction.

7. Quick comparison

<code>Animal a;Animal* a = new Animal();</code>	What is <code>a</code> ?	Object	Pointer	Object location	Usually stack/local storage	Dynamic storage
(heap)	Access <code>a.sound()a->sound()</code>	Destruction	Automatic	Manual with <code>delete</code>	Memory leak possible?	NoYes, if mismanaged
Runtime polymorphism	Can use references	Commonly used with pointers	Modern default	☒ Yes	☒ Yes	Usually avoid raw <code>new</code>

Mental model

Remember just this:

```
Animal a;
```

means:

```
"Give me an Animal object."
```

Whereas:

```
Animal* a = new Animal();
```

means:

```
"Create an Animal somewhere dynamically, and give me its address."
```

And this:

```
Animal* a = new Dog();
```

means:

```
"Create a Dog, but let me interact with it through the Animal interface."
```

That last one is the key to understanding **runtime polymorphism**.